

MÓDULO 10: Acceso a Datos Repositoriales vía APIs y Protocolo OAI-PMH (Digital.CSIC)

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 10 - Web APIs, REST Architecture, OAI-PMH Protocol, XML Parsing with BeautifulSoup & Digital.CSIC Harvesting

1. Objetivos de Aprendizaje

- **APIs Web y Arquitectura REST vs. OAI-PMH:** Comprender la diferencia entre las APIs RESTful modernas (JSON, endpoints por recurso) y el protocolo de recolección de metadatos académicos **OAI-PMH** (XML, verbos HTTP).
- **Acceso a Repositorios Institucionales:** Explorar cómo **Digital.CSIC** (basado en la plataforma **DSpace**) expone metadatos científicos normalizados en esquema **Dublin Core** (`oai_dc`).
- **Verbos OAI-PMH:** Dominar los verbos fundamentales (`Identify` , `ListMetadataFormats` , `ListIdentifiers` , `ListRecords` , `GetRecord`) y el manejo de paginación con `resumptionToken` .
- **Parseo de XML con BeautifulSoup:** Utilizar `requests` y `BeautifulSoup(response.content, 'xml')` para la inspección y extracción de etiquetas XML y namespaces.
- **Búsqueda y Extracción Programática:** Desarrollar scripts en Python para listar, filtrar por palabras clave, extraer autores (`dc:creator`), títulos (`dc:title`) y handles institucionales (`10261/...`).
- **Integración con la Librería del Curso (`lib/digital_csic.py`):** Conectar los conceptos teóricos y prácticos aprendidos con las funciones automatizadas del paquete de utilidades del curso.

2. Fundamentos Teóricos: APIs REST vs. Protocolo OAI-PMH

¿Qué es una API RESTful?

Una **API REST** (*Representational State Transfer*) es una interfaz de programación de aplicaciones basada en los principios de la web:

- **Recursos Identificados por URIs:** Cada entidad (usuario, publicación, proyecto) tiene una dirección de red propia.
- **Métodos HTTP:** Operaciones CRUD mediante `GET` (lectura), `POST` (creación), `PUT / PATCH` (actualización) y `DELETE` (eliminación).
- **Formatos Ligeros:** Respuestas estandarizadas habitualmente en formato **JSON**.
- **Autenticación y Control:** Autenticación por Tokens (**API Keys**, **OAuth 2.0**) y políticas de límite de frecuencia (*rate limiting*).

¿Qué es el Protocolo OAI-PMH?

OAI-PMH (*Open Archives Initiative — Protocol for Metadata Harvesting*) es un estándar internacional de bajo umbral diseñado específicamente para la **recolección masiva e interoperable de metadatos** entre repositorios académicos e instituciones de investigación.

- **Transporte y Formato:** Funciona sobre HTTP `GET / POST` y responde en formato estructurado **XML**.
- **Enfoque en Metadatos:** Recolecta fichas bibliográficas normalizadas (título, autor, fecha, identificador Handle/DOI), no archivos binarios pesados (PDFs/ZIPs).
- **Esquema Dublin Core (`oai_dc`):** Todos los repositorios compatibles con OAI-PMH deben soportar por mandato el esquema Dublin Core no calificado.



Cuadro Comparativo: REST API vs. OAI-PMH

Característica	API REST Modernas	Protocolo OAI-PMH
Propósito Principal	Interacción transaccional y consultas específicas.	Recolección masiva de metadatos (<i>Harvesting</i>).
Formato de Respuesta	Principalmente JSON (o XML/YAML).	Estrictamente XML estructurado con Namespaces.
Parámetros de Consulta	Filtros arbitrarios (<code>?q=ciencia&page=1</code>).	Verbos estandarizados (<code>Identify</code> , <code>ListRecords</code>).
Paginación	Parámetros <code>page</code> , <code>offset</code> , <code>limit</code> .	Controlada por el servidor mediante <code>resumptionToken</code> .
Autenticación	API Key, Bearer Token, OAuth.	Generalmente abierto (Acceso público de Ciencia Abierta).

3. Esquema de Metadatos Dublin Core (oai_dc)

El protocolo OAI-PMH exige que todo repositorio exponga sus metadatos al menos en el formato **Dublin Core no calificado** (oai_dc). Este estándar define 15 elementos fundamentales de metadatos:

- `dc:title` → Título de la obra o artículo científico.
- `dc:creator` → Autor(es) o entidad creadora.
- `dc:date` → Fecha de publicación o depósito.
- `dc:identifier` → Identificador único (Handle `10261/...`, DOI, URI).
- `dc:description` → Resumen (*abstract*) o descripción de la investigación.
- `dc:subject` → Palabras clave, materias o descriptores temáticos.
- `dc:publisher` → Editorial o institución editora.
- `dc:type` → Tipo de recurso (artículo, tesis, conjunto de datos, patente).
- `dc:language` → Idioma del documento (ej. `es`, `en`).

4. Primer Acceso a Digital.CSIC: Verbo `Identify`

Digital.CSIC expone su servicio OAI-PMH en la dirección base: 📍

`https://digital.csic.es/dspace-oai/request`

A continuación realizamos una consulta inicial con el verbo **`Identify`**, que devuelve los metadatos generales del repositorio.

```
In [ ]: # Importación de librerías para comunicación HTTP, parseo XML y ruta de sist
import os
import sys
import requests
from bs4 import BeautifulSoup
```

```
In [ ]: # Añadir el directorio raíz del proyecto a sys.path para acceder al paquete
root_path = os.path.abspath("..") if os.path.exists("../lib") else os.path.a
if root_path not in sys.path:
    sys.path.append(root_path)

import lib.digital_csic as dcsic
from lib.mylib import linea as linea
# Ruta base flexible para el directorio DATASETS
datasets_dir = "../DATASETS" if os.path.exists("../DATASETS") else "DATASETS
linea()
print(f"Librería digital_csic cargada correctamente.")
print(f"Directorio de datos activo: {datasets_dir}")
linea()
```

```
In [ ]: # Endpoint base del servicio OAI-PMH de Digital.CSIC
base_url = "https://digital.csic.es/dspace-oai/request"
```

```

# Parámetros HTTP: Verbo Identify (Información general del repositorio)
params = {"verb": "Identify"}

# Realizamos la petición HTTP GET al servidor de Digital.CSIC
response = requests.get(base_url, params=params)

if response.status_code == 200:
    # Parseamos la respuesta XML utilizando BeautifulSoup con el motor 'xml'
    soup = BeautifulSoup(response.content, "xml")

    repo_name = soup.find("repositoryName").text if soup.find("repositoryName") else "No se encontró el nombre del repositorio"
    url_base = soup.find("baseURL").text if soup.find("baseURL") else "No se encontró la URL base"
    proto_ver = soup.find("protocolVersion").text if soup.find("protocolVersion") else "No se encontró la versión del protocolo"
    admin_email = soup.find("adminEmail").text.strip() if soup.find("adminEmail") else "No se encontró el email del administrador"

    linea()
    print(f"🏠 Nombre del Repositorio: {repo_name}")
    linea()
    print(f"🔗 Base URL OAI-PMH: {url_base}")
    linea()
    print(f"📄 Versión Protocolo OAI-PMH: {proto_ver}")
    linea()
    print(f"✉ Email del Administrador: {admin_email}")
    linea()
else:
    print(f"❌ Error al consultar el endpoint OAI-PMH. Código HTTP: {response.status_code}")

```

5. Recuento Total de Registros y Paginación (ListRecords + resumptionToken)

Para consultar el volumen total de publicaciones custodiadas en Digital.CSIC, utilizamos el verbo **ListRecords** especificando el formato `oai_dc`. La respuesta incluye la etiqueta `<resumptionToken>` con el atributo `completeListSize`.

```

In [ ]: # Petición para listar registros y consultar el volumen total en Digital.CSIC
from datetime import datetime
import time

params_records = {
    "verb": "ListRecords",
    "metadataPrefix": "oai_dc"
}

# Breve pausa de cortesía en peticiones de red
time.sleep(1)

resp_records = requests.get(base_url, params=params_records)

if resp_records.status_code == 200:
    soup_records = BeautifulSoup(resp_records.content, "xml")

```

```

# Inspeccionar las primeras 5 líneas del texto XML recibido
lineas_xml = soup_records.text.splitlines()
print("📄 Inspección de las primeras 5 líneas del XML recibido:")
for idx, linea_text in enumerate(lineas_xml[:5], start=1):
    print(f"  [{idx}]: {linea_text}")

# Extracción del tag resumptionToken (contiene el atributo completeListSize)
resumption_tag = soup_records.find("resumptionToken")

if resumption_tag and resumption_tag.has_attr("completeListSize"):
    total_records = int(resumption_tag["completeListSize"])
else:
    records_page = soup_records.find_all("record")
    total_records = len(records_page)

fecha_hoy = datetime.now().strftime("%d/%m/%Y")
linea()
print(f"📅 A fecha {fecha_hoy}, el número total de registros en Digital.
linea()
else:
    print(f"❌ Error HTTP {resp_records.status_code} al solicitar ListRecords")

```

6. Búsqueda y Filtrado Programático por Palabras Clave

Dado que el protocolo OAI-PMH no ofrece un buscador textual por palabra clave en la URL, procesamos en memoria las etiquetas `<record>` devueltas, inspeccionando `dc:title`, `dc:creator` e `identifier`.

```

In [ ]: # Extracción de metadatos y filtrado por palabra clave en títulos
registros = soup_records.find_all("record")
keyword = "csic"

ids_list = []
authors_list = []
titles_list = []

for r in registros:
    titulo_tag = r.find("dc:title")
    identifier_tag = r.find("identifier")
    autores_tags = r.find_all("dc:creator")

    if titulo_tag and identifier_tag and autores_tags:
        titulo_text = titulo_tag.text.strip()

        # Filtrar si la palabra clave está en el título (case-insensitive)
        if keyword in titulo_text.lower():
            # Extraer Handle (ej. oai:digital.csic.es:10261/1400 -> 10261/1400)
            oai_id = identifier_tag.text.strip()
            handle = oai_id.split(":")[-1] if ":" in oai_id else oai_id

            # Formatear lista de autores con sus índices [1], [2]
            nombres_autores = [a.text.strip() for a in autores_tags]

```

```

        autores_str = ", ".join([f"{nom}[{i+1}]" for i, nom in enumerate(
            ids_list.append(handle)
            authors_list.append(autores_str)
            titles_list.append(titulo_text)

linea()
print(f"🔍 Registros encontrados con la palabra '{keyword}' en el primer lote")
linea()

# Iteración paralela con zip y enumerate
for idx, (h, aut, tit) in enumerate(zip(ids_list, authors_list, titles_list)):
    print(f"[{idx}] Handle: {h}")
    print(f"    Autores: {aut}")
    print(f"    Título: {tit}")
    linea()

```

7. Integración con la Librería del Curso (lib/digital_csic.py)

La librería del curso encapsula estas peticiones OAI-PMH en funciones reutilizables. A continuación verificamos cómo `dcsic.fetch_digital_csic_record()` consulta el servidor en tiempo real.

```

In [ ]: # Demostración del uso de la librería personalizada lib/digital_csic.py
import lib.digital_csic as dcsic
from lib.mylib import linea

# Consultar un Handle específico en Digital.CSIC mediante la librería
sample_handle = "10261/403880"
record_info = dcsic.fetch_digital_csic_record(sample_handle)

# Comprobación de existencia/validez (devuelve dict si existe, None si no)
es_valido = record_info is not None

linea()
print(f"📄 Registro recuperado vía lib.digital_csic:")
print(f" - Handle:      {sample_handle}")
print(f" - ¿Existe?:     {es_valido}")

if record_info:
    print(f" - Título:      {record_info.get('title')}")
    print(f" - Autores:     {record_info.get('authors')}")
    print(f" - Año:         {record_info.get('year')}")
    print(f" - Fuente:      {record_info.get('journal')}")
    print(f" - DOI:         {record_info.get('doi')}")
linea()

```

 Generación de Citas Bibliográficas con `GeneraAPA()`

La librería `lib/digital_csic.py` proporciona la función **GeneraAPA(response)**, diseñada para extraer automáticamente los metadatos bibliográficos de un registro obtenido desde el servidor OAI-PMH de **Digital.CSIC** y formatearlos en una referencia estructurada.

📌 Funcionamiento y Parámetros

- **Función:** `dcsic.GeneraAPA(response)`
- **Parámetro response:** Recibe la respuesta de `requests.get()` al consultar un registro mediante el verbo `GetRecord (metadataPrefix="oai_dc")`. (También acepta cadenas de texto XML o instancias de `BeautifulSoup`).
- **Información extraída:**
 1. **Autores (dc:creator):** Une la lista de autores separados por comas.
 2. **Título (dc:title):** Extrae el título de la publicación científica.
 3. **Revista / Fuente (identifier):** Identifica heurísticamente la cita de la fuente.
 4. **DOI:** Extrae la dirección `https://doi.org/...` o el prefijo `10.XXXX/...`.
 5. **Handle:** Obtiene la dirección permanente `http://hdl.handle.net/10261/...`.

```
In [ ]: from lib.digital_csic import GeneraAPA as GeneraAPA
# 1. Petición OAI-PMH GetRecord a Digital.CSIC
base_url = "https://digital.csic.es/dspace-oai/request"
params = {
    "verb": "GetRecord",
    "metadataPrefix": "oai_dc",
    "identifier": f"oai:digital.csic.es:{sample_handle}"
}

response = requests.get(base_url, params=params, timeout=10)

# 2. Generar cita formateada llamando a la librería
cita_apa = GeneraAPA(response)

linea()
print("Cita Bibliográfica (Formato APA / Digital.CSIC):")
print(cita_apa)
linea()
```